

Comparative Analysis of Learned and Designed Features:

A Deep Dive into MusicNN and Scattering Transform for Music Instrument Recognition

NICLAS LÖFVENMARK

Degree Programme in Engineering Physics
Date: April 14, 2026

Supervisor: Bob Sturm
Examiner: André Holzapfel
School of Electrical Engineering and Computer Science

Abstract

In the realm of automated music analysis, the task of music instrument recognition stands as a pivotal challenge. As digital music libraries expand, the need for optimal features—those that can succinctly and effectively represent musical nuances—becomes paramount. This thesis delves into this challenge by juxtaposing two distinct paradigms of feature extraction: the meticulously designed Scattering Transform and the learned penultimate features from MusicNN models (MSD and MTT). The primary challenge addressed is the comparative performance of designed versus learned features in music instrument recognition, a problem of paramount importance given its implications for the broader field of music informatics.

The significance of this exploration is underscored by the direct impact of feature choice on the accuracy and efficiency of recognition systems. With the burgeoning size of digital music libraries, effective recognition can revolutionize tasks like music recommendation, cataloging, and analysis. The inherent complexity of musical compositions and the diverse nature of musical instruments have made this a persistent challenge. Leveraging the ESSID dataset and a custom-curated dataset of 70 tracks from YouTube, classifiers such as KNN and SVM were employed to evaluate the efficacy of these features.

Results revealed a marked enhancement in recognition accuracy when integrating both designed and learned features. This suggests that they might encapsulate complementary musical information, paving the way for future studies to delve deeper into their inherent attributes. The observed synergy between designed and learned features suggests that their combined use might be pivotal for enhanced music instrument recognition. Furthermore, the introduction of a novel dataset sourced from YouTube emphasizes the potential of incorporating real-world, diverse data in music instrument recognition tasks, setting the stage for future research in this domain.

Keywords

Music Instrument Recognition, Scattering Transform, MusicNN, Feature Extraction, Designed Features, Learned Features.

Sammanfattning

Inom området för automatiserad musikanalys utgör uppgiften att känna igen musikinstrument en central utmaning. I takt med att digitala musikbibliotek växer ökar behovet av lämpliga representationer – sådana som på ett kortfattat och effektivt sätt kan representera musikaliska nyanser. Denna avhandling fördjupar sig i denna utmaning genom att jämföra två distinkta paradigmer för egenskapsutvinning: den noggrant utformade Spridningstransformen och de inlärda penultimate egenskaperna från MusicNN-modeller (MSD och MTT). Den primära utmaning som behandlas är prestandan hos utformade kontra inlärda funktioner vid identifiering av musikinstrument, en problematik av stor betydelse med tanke på dess implikationer för det bredare området musikinformatik.

Betydelsen av denna utforskning understryks av den direkta påverkan som val av funktion har på noggrannheten och effektiviteten hos igenkännande system. Med den växande storleken på digitala musikbibliotek kan effektiv igenkänning revolutionera uppgifter som musikrekommendation, katalogisering och analys. Den inneboende komplexiteten hos musikaliska kompositioner och den mångfacetterade naturen hos musikinstrument har gjort detta till en ihållande utmaning. Genom att utnyttja ESSID-datasetet och ett skräddarsytt dataset med 70 spår från YouTube användes klassificerare som KNN och SVM för att utvärdera effektiviteten hos dessa funktioner.

Resultaten visade en tydlig förbättring av igenkännande-noggrannheten när både utformade och inlärda funktioner integrerades. Detta tyder på att de kanske fångar kompletterande musikalisk information, vilket banar väg för framtida studier att djupdyka i deras inneboende egenskaper. Den observerade synergins mellan utformade och inlärda funktioner föreslår att deras gemensamma användning kan vara avgörande för en förbättrad igenkänning av musikinstrument. Vidare betonar införandet av ett nytt dataset hämtat från YouTube potentialen att inkludera verkliga, mångfaldiga data i uppgifter om igenkänning av musikinstrument, vilket skapar grunden för framtida forskning inom detta område.

Nyckelord

Identifiering av Musikinstrument, Spridningstransform, MusicNN, Funktionsextraktion, Utformade Funktioner, Inlärda Funktioner.

Acknowledgments

I would like to extend my heartfelt gratitude to the following individuals, without whom this thesis would not have been possible:

First and foremost, I express my sincere appreciation to my supervisor, Professor Bob Sturm. His exceptional guidance, unwavering support, and profound expertise in music informatics have been instrumental in shaping this research project. I am deeply grateful for his mentorship and for introducing me to the fascinating world of music informatics.

I am also thankful to Professor André Holzapfel for co-instructing the course in music informatics that sparked my interest in this field. The insights and knowledge gained from that course have provided the foundation for my thesis work.

I extend my heartfelt thanks to my classmates who have been my companions on this academic journey. Your camaraderie, discussions, and shared experiences have enriched my learning experience and made the challenges more manageable.

To my family, I am grateful for your unwavering support, encouragement, and understanding throughout my academic pursuits.

Last but not least, I want to acknowledge the broader academic community and the resources it offers, which have been invaluable in shaping my understanding and contributing to the development of this thesis.

Stockholm, August 2023

Niclas Löfvenmark

Contents

1	Introduction	1
2	Literature Survey	3
2.1	Overview of Previous Work	3
2.2	Comparison of Different Approaches	3
2.3	Gaps in Current Literature	3
3	Background and Theory	4
3.1	Signal Processing	4
3.1.1	Introduction to Signal Processing	4
3.1.2	The Fourier Transform	5
3.1.3	The Wavelet Transform: Localizing the Fourier Transform	5
3.1.4	Scattering Transform: Hierarchical Wavelet Analysis	6
3.1.5	Deep Scattering Spectrum: A Practical Application	7
3.1.6	Summary	8
3.2	Feature Selection - TEMPORARY LOCATION OF SECTION	9
3.2.1	Filter Methods	9
3.2.2	Wrapper Methods	12
3.2.3	Embedded Methods	13
3.3	Feature Representations	14
3.3.1	Introduction	14
3.3.2	Designed vs. Learned Feature Representations	14
3.3.3	Scattering Transform - We kinda talked already about this... Repetitive ?	14
3.3.4	MusicNN - Kinda short though? Perhaps longer more detail explanation?	15
3.3.5	Summary	15
3.4	Machine Learning Methods	15
3.4.1	Introduction	15

3.4.2	Support Vector Machines (SVM)	15
3.4.3	k-Nearest Neighbors (KNN)	17
3.4.4	Random Forest	18
3.4.5	Summary	19
3.5	Dimensionality Reduction Techniques	19
3.5.1	Principal Component Analysis (PCA)	19
3.5.2	Other Dimensionality Reduction Techniques	20
4	Method or Methods	21
4.1	Model Training and Evaluation	21
4.2	Essid Dataset	21
5	What you did	23
5.1	Data Transformation Pipeline	23
5.1.1	Overview	23
5.1.2	Scattering Transform	23
5.1.3	MusicNN Transforms	24
5.2	Experiments	24
6	Results and Analysis	25
6.1	Dimensionality Reduction Using Principal Component Analysis	29
7	Discussion	31
8	Conclusions and Future work	33
9	Something Extra	34

List of Figures

6.1	KNN Performance on PF_MSD_ESSID Dataset	25
6.2	KNN Performance on MSD_ESSID Dataset	26
6.3	SVM Performance on MSD_ESSID Dataset	26
6.4	KNN Performance on sx mean features	26
6.5	SVM Performance on sx mean features	26
6.6	KNN Performance on combined features, sxmean and mtt . . .	27
6.7	SVM Performance on combined features, sxmean and mtt . . .	27
6.8	SVM on msd (youtube), for 200 dim space	27
6.9	SVM on msd (youtube), for 200 dim space, but projected onto the pca of the msd space	27
6.10	SVM on combined (youtube dataset), for 573 dim space	28
6.11	SVM on combined (youtube dataset), for 200 dim space. So this is projected onto the 200 PCs of the svm essid space . . .	28

List of Tables

- 4.1 Overview of the Essid dataset 22
- 4.2 Overview of the Youtube Excerpts Dataset 22

Listings

If you have listings in your thesis.

The list of acronyms and abbreviations should be in alphabetical order based on the spelling of the acronym or abbreviation.

Chapter 1

Introduction

Music instrument recognition is a task that aims to automatically identify the instruments playing in a given audio recording. This task is a fundamental problem in the field of Music Information Retrieval (MIR), where the goal is to extract meaningful information from music data.

The ability to automatically recognize musical instruments in audio recordings is of significant value in various applications. For instance, it can enable enhanced music browsing experiences, where users can search and organize music based on the instruments they contain. This metadata can also be used to improve music education tools, assist in music production, and contribute to research in musicology. Moreover, instrument recognition can be a critical component in more complex systems such as automatic music transcription and source separation.

Recognizing musical instruments in audio signals is a challenging task due to the complex and highly variable nature of musical sounds. Instruments can produce sounds with a wide range of timbres and pitches, and their sonic characteristics can change based on playing techniques and recording conditions.

A central aspect of music instrument recognition is the choice of feature representations used to describe audio signals. These features should capture the relevant information in the audio signal that allows different instruments to be distinguished from each other. One approach to feature extraction involves designing features based on our understanding of music and audio signals. A notable example of this approach is the Scattering Transform,

a time-frequency representation that extends wavelet transforms to capture higher-order modulation spectra in signals. Introduced by Joakim Andén and Stéphane Mallat [?], the Scattering Transform has shown promise in various audio signal processing tasks due to its ability to capture translation-invariant and stable representations of signals.

An alternative approach to feature extraction is to learn features directly from data using machine learning techniques. One such method is MusicNN, a set of pre-trained deep convolutional neural networks for music tagging, which can be used to extract features from audio signals [?]. Specifically, this thesis utilizes the penultimate layer of MusicNN, which provides a rich and discriminative representation of the audio signal that is learned during the training of the neural network.

The primary goal of this thesis is to study and compare the effectiveness of designed features, specifically the Scattering Transform, and learned features, specifically those extracted using MusicNN, in the context of solo instrument recognition. A key question this thesis aims to answer is whether these two types of features contain complementary information. By concatenating these two types of features, we not only explore the potential for improved instrument recognition performance but also investigate whether designed and learned features capture different aspects of the audio signal.

The remainder of this thesis is organized as follows: Chapter 2 provides the necessary background on music instrument recognition, feature representations, and related work. Chapter 3 describes the methods and experimental setup used in this study. Chapter 4 presents the results of our experiments and provides a detailed analysis. Chapter 5 discusses the implications of our findings, and Chapter 6 concludes the thesis and suggests directions for future work.

Chapter 2

Literature Survey

2.1 Overview of Previous Work

2.2 Comparison of Different Approaches

2.3 Gaps in Current Literature

Chapter 3

Background and Theory

3.1 Signal Processing

This section provides the theoretical background necessary for understanding the scattering transform. We begin with the foundational Fourier Transform, discuss its limitations, and introduce subsequent developments that address these limitations, culminating in the scattering transform.

3.1.1 Introduction to Signal Processing

Signal processing is a field of study that focuses on the analysis, manipulation, and interpretation of signals. Signals, in this context, refer to time-varying quantities that carry information, such as audio recordings, images, or sensor data. The primary goal of signal processing is to extract meaningful information from these signals or to transform them in a useful way.

One of the fundamental tools in signal processing is the Fourier Transform. It allows us to decompose a signal into its constituent frequencies, providing a different perspective on the signal. This frequency perspective is often more insightful and actionable than the raw time-domain signal, especially for analyzing the harmonic content of sounds or finding patterns in data.

In this section, we will start with the Fourier Transform, explore its limitations, and then introduce subsequent developments that address these limitations. We will culminate with the Scattering Transform, a modern and powerful tool that builds upon these foundational concepts to provide stable and informative

representations of signals.

3.1.2 The Fourier Transform

The Fourier Transform, named after Jean-Baptiste Joseph Fourier, is a mathematical tool that decomposes a function into its constituent frequencies, also known as its frequency spectrum. It is a fundamental tool in signal processing and allows us to analyze and manipulate signals in the frequency domain. Mathematically, the Fourier Transform of a continuous and integrable function $f(t)$ is defined as:

$$\mathcal{F}\{f(t)\} = F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-j\omega t} dt \quad (3.1)$$

However, the Fourier Transform has a significant limitation: it does not capture the time-localized characteristics of a signal. This is because the Fourier Transform provides a global representation of the signal, analyzing the entire signal as a whole, rather than considering its local (time-specific) characteristics. In the Fourier Transform, the integral is taken over all time, from $-\infty$ to ∞ , and there is no time variable in $F(\omega)$. This means that while $F(\omega)$ tells us the amplitude of the frequency components ω in the signal, it does not tell us when these frequency components occur in the signal. This limitation is significant when analyzing signals with non-stationary or time-varying characteristics, where the when is as important as the what.

3.1.3 The Wavelet Transform: Localizing the Fourier Transform

The Wavelet Transform, introduced to address the limitations of the Fourier Transform, enables a multi-resolution analysis of signals. It provides a windowed view of different parts of the signal, which is essential for analyzing signals with non-stationary or time-varying characteristics.

Mathematically, the wavelet transform of a signal $x(t)$ is defined as:

$$Wx(a, b) = \frac{1}{\sqrt{|a|}} \int x(t)\psi^*\left(\frac{t-b}{a}\right) dt \quad (3.2)$$

where:

- $\psi(t)$ is the wavelet function, also known as the mother wavelet

- a is the scale parameter
- b is the translation parameter
- ψ^* denotes the complex conjugate of ψ

This wavelet transform is designed to cover the whole frequency axis, ensuring that it is a stable and invertible operator. Unlike the Fourier Transform, which provides a global representation of the entire signal, the wavelet transform retains the time information, making it possible to analyze signals at different resolutions or scales.

This is particularly beneficial when the signal has abrupt changes or non-stationary features, which are common in real-world signals. The wavelet transform, therefore, offers a powerful tool for signal analysis, allowing us to understand both the frequency content and the time localization of features within a signal.

3.1.4 Scattering Transform: Hierarchical Wavelet Analysis

Building upon the Wavelet Transform, we introduce the Scattering Transform—a hierarchical extension that further refines our ability to analyze and represent signals. This transform inherits the time-frequency localization of wavelets and adds invariant and stable representations, making it particularly powerful for complex signal analysis.

Mathematical Definition

The Scattering Transform of a signal $x(t)$, denoted as Sx , is computed through a cascade of wavelet transforms and modulus operators. For a wavelet $\psi(t)$ and a scaling function $\phi(t)$, the m -th order scattering coefficient is defined as:

$$S_m x(t) = \int x(t) \psi_{j_1} * \cdots * \psi_{j_m} * \phi(t) dt \quad (3.3)$$

where $*$ denotes convolution, and $j_1, \dots, j_m$ are the scales of the wavelets.

Key Properties

The Scattering Transform possesses several notable properties:

- **Translation Invariance:** It provides stable representations that are largely invariant to translations in the input signal.
- **Stability to Deformations:** The transform is stable under time-warping deformations, making it robust to small distortions in the signal.
- **Preservation of High Frequencies:** Unlike traditional wavelet transforms, scattering retains high-frequency information, which is crucial for signal discrimination.

Advantages Over Simple Wavelet Transforms

While wavelet transforms excel in providing time-frequency representations of signals, the Scattering Transform further excels by offering:

- A multi-layered, deep architecture that captures complex and hierarchical patterns in data.
- Invariance to common signal transformations, which is essential for robust signal classification and analysis.

This hierarchical and invariant nature of the Scattering Transform makes it a powerful tool, especially in contexts where the data exhibits complex and multifaceted structures, such as in music and audio signal analysis.

3.1.5 Deep Scattering Spectrum: A Practical Application

Having established the theoretical foundation of the Scattering Transform, we now turn to a practical application of this theory: the Deep Scattering Spectrum. This section illustrates how the Scattering Transform can be effectively employed in real-world scenarios, particularly in the context of music and audio analysis.

The Deep Scattering Spectrum is a representation of a signal that is invariant to translation and stable to deformations, making it highly discriminative for various audio classification tasks. It is computed by applying the Scattering Transform to an input signal and then organizing the resulting scattering coefficients into a structured representation.

$$DSS(x) = Sx(j_1, j_2, \dots, j_m) \quad (3.4)$$

where:

- $DSS(x)$ denotes the Deep Scattering Spectrum of the signal x
- $Sx(j_1, j_2, \dots, j_m)$ are the scattering coefficients of the signal x , as computed by the Scattering Transform

Applications in Music and Audio Analysis The Deep Scattering Spectrum has found substantial applications in various music and audio analysis tasks, including:

- **Genre Classification:** The DSS can be used to create highly discriminative features for classifying music into different genres.
- **Speech Recognition:** The DSS provides a robust representation of speech signals, making it useful for automatic speech recognition systems.
- **Sound Source Separation:** The DSS can be employed to separate different sources in a complex audio scene.

These applications demonstrate the power and versatility of the Deep Scattering Spectrum as a tool for practical, real-world signal analysis.

3.1.6 Summary

To conclude this section, we have journeyed from the foundational Fourier Transform, which serves as a cornerstone in signal processing, to the Wavelet Transform, which addresses the Fourier Transform's limitations by providing time-frequency localization. We have then explored the Scattering Transform, a hierarchical extension of wavelet analysis that offers invariant and stable representations of signals. This powerful tool is particularly adept at handling complex and multifaceted structures in data, as demonstrated by its practical application in the form of the Deep Scattering Spectrum.

- The **Fourier Transform** is the foundational tool that decomposes a signal into its constituent frequencies but lacks time localization.
- The **Wavelet Transform** addresses this limitation, offering a multi-resolution, time-frequency representation of signals.
- The **Scattering Transform** builds upon wavelets to provide a hierarchical, invariant, and stable representation of signals, making it particularly suited for complex signal analysis.

- The **Deep Scattering Spectrum** is a practical application of the Scattering Transform, demonstrating its effectiveness and versatility in real-world scenarios, particularly in music and audio analysis.

3.2 Feature Selection - TEMPORARY LOCATION OF SECTION

Feature selection is an essential step in music instrument recognition to improve model performance, reduce computational complexity, and enhance interpretability, i.e., the extent to which cause and effect can be observed within a given system. Several commonly employed feature selection techniques were considered in this thesis:

3.2.1 Filter Methods

Filter methods are a category of feature selection techniques that assess the relevance of features based on their individual characteristics, independent of any specific machine learning algorithm. These methods evaluate the inherent properties of features, such as their statistical measures or correlation with the target variable, to determine their importance. Filter methods are computationally efficient and provide a quick way to identify potentially informative features.

Correlation-based Feature Selection (CFS)

Correlation-based Feature Selection (CFS) evaluates the relationship between each feature and the target variable. It calculates the correlation score between each feature and the target, as well as the correlation among features. CFS uses a subset evaluation criterion that aims to select features that are highly correlated with the target variable while having low intercorrelation among themselves. By considering both relevance and redundancy, CFS seeks to find the most informative and non-redundant feature subset.

CFS is particularly useful when dealing with high-dimensional datasets, as it helps identify the most influential features for predicting the target variable. However, CFS may overlook complex interactions among features and could potentially miss important feature combinations.

Chi-Square Test

The Chi-Square Test measures the dependence between categorical features and a categorical target variable. It calculates the chi-square statistic, which compares the observed frequencies of feature-target associations with the expected frequencies under the assumption of independence. Features with higher chi-square statistics indicate a stronger association with the target variable.

The Chi-Square Test is widely used in feature selection for classification tasks with categorical data. It is especially effective when dealing with categorical features and categorical target variables, such as text classification or sentiment analysis. However, it is not suitable for continuous or numerical features.

Information Gain

Information Gain is a feature selection method based on entropy and information theory. It quantifies the amount of information gained about the target variable when a specific feature is known. Features with higher information gain have a greater ability to discriminate between different classes or categories of the target variable.

Information Gain is commonly used in decision trees and can be applied to both categorical and numerical features. It is particularly useful for classification tasks and helps identify features that contribute the most to the decision-making process. However, Information Gain tends to favor features with high cardinality or a large number of unique values.

ANOVA (Analysis of Variance)

ANOVA is a statistical technique used to assess the differences between group means. In feature selection, ANOVA evaluates the variance between the means of different classes or categories of the target variable for each feature. It calculates the F-value, which represents the ratio of between-group variance to within-group variance. Features with higher F-values indicate a greater discrimination power.

ANOVA is widely applied in feature selection for classification tasks and is suitable for both numerical and categorical features. It is effective in identifying features that exhibit significant variations across different classes

or categories of the target variable. However, ANOVA assumes that the feature distributions are approximately normal and may not perform well with skewed or non-normal data.

Mutual Information

Mutual information is a measure of the dependency between two variables. It quantifies the amount of information obtained about one variable through observing the other variable. In the context of feature selection, mutual information can be used to measure the dependency between each feature and the target variable. Features that have high mutual information with the target variable are considered more relevant for prediction.

The mutual information between two variables X and Y can be defined as:

$$I(X; Y) = \sum_{y \in Y} \sum_{x \in X} p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right) \quad (3.5)$$

where $p(x, y)$ is the joint probability mass function of X and Y , and $p(x)$ and $p(y)$ are the marginal probability mass functions of X and Y respectively.

In feature selection, we calculate the mutual information between each feature and the target variable, and select the features that have the highest mutual information.

K-Best Feature Selection

K-Best Feature Selection is a simple yet effective filter method that selects the top K features based on statistical measures. It assigns a score or ranking to each feature using statistical tests such as the aforementioned ANOVA, mutual information, or chi-square. The K features with the highest scores are retained, while the rest are discarded.

K-Best Feature Selection is versatile and can be applied to both classification and regression tasks. It allows for the selection of the most relevant features based on their individual predictive power. However, K-Best Feature Selection does not consider the interactions or dependencies between features, which may lead to the exclusion of important feature combinations.

K-Best Feature Selection is particularly useful when the number of available features is large, and there is a need to reduce the dimensionality of the dataset while preserving the most informative features. The choice of K depends on factors such as the dataset size, complexity, and the desired balance between model performance and computational efficiency.

By selecting the top K features based on statistical measures, we aim to prioritize those features that contribute the most to the classification accuracy.

3.2.2 Wrapper Methods

Wrapper methods utilize machine learning algorithm in order to evaluate the importance of features. The evaluation involves evaluating a given algorithm on a subset of features and subsequently select the subset that provides the best performance. In contrast to filter methods, wrapper methods consider interactions between features. Consequently, wrapper methods often perform better, albeit at the cost of higher computational cost. The most common wrapper methods include forward selection, backward selection, stepwise selection, Sequential Forward Selection (SFS), and Recursive Feature Elimination (RFE).

Forward Selection

Forward selection is a simple and intuitive method of feature selection. It starts with no features and adds one feature at a time. At each step, it adds the feature that gives the best improvement to the model. The process stops when all features have been added.

Backward Selection

Backward selection is the opposite of forward selection. It starts with all features and removes one feature at a time. At each step, it removes the feature whose removal gives the best improvement to the model. The process stops when no features can be removed without worsening the model performance.

Stepwise Selection

Stepwise selection is a combination of forward and backward selection. It starts with no features and adds one feature at a time, like forward selection. However, after adding a new feature, it checks whether any of the already

included features can be removed without worsening the model performance, like backward selection. The process stops when no features can be added or removed.

Sequential Forward Selection (SFS)

Sequential Forward Selection (SFS) is a type of forward selection. It starts with no features and adds one feature at a time. At each step, it selects the feature that gives the best improvement to the model. Unlike simple forward selection, SFS has a stopping criterion based on model performance. The process stops when the addition of new features does not improve the performance of the model.

Recursive Feature Elimination (RFE)

Recursive Feature Elimination (RFE) is similar to backward selection. It starts with all features and removes one feature at a time. However, unlike simple backward selection, RFE uses a machine learning model to rank features by importance, removes the least important features, and re-fits the model. This process is repeated until a specified number of features remains.

3.2.3 Embedded Methods

Embedded methods integrate the feature selection process into the model training process. They are typically specific to certain machine learning algorithms and are more efficient than wrapper methods as they avoid the need to retrain the model for each subset of features.

L1 Regularization (Lasso)

L1 Regularization, also known as Lasso, is a technique that introduces a penalty term based on the absolute values of feature coefficients during model training. This penalty encourages sparsity in the feature space, forcing less relevant features to have zero or close-to-zero coefficients. Features with non-zero coefficients are considered important and retained for further analysis.

L1 Regularization is commonly used in linear models and is effective in handling high-dimensional datasets. It automatically selects a subset of features by shrinking the coefficients of irrelevant features towards zero. L1 Regularization is particularly useful when dealing with highly correlated features or when interpretability is a priority.

Tree-based Feature Selection

Tree-based models, such as decision trees, random forests, and gradient boosting machines, can also be used for feature selection. These models provide a measure of feature importance based on the number of times a feature is used to split the data across all trees. Features that are frequently used for splitting are considered important. This method is particularly useful when dealing with non-linear and interactive feature-to-target relationships.

3.3 Feature Representations

3.3.1 Introduction

Feature representations transform raw, complex data into a more manageable and interpretable format for machine learning algorithms. They act as a bridge between the raw data and the algorithm, translating intricate data patterns into a form that is more digestible for computers. This transformation is akin to translating a complex novel into a summary that retains the key points, making it quicker and easier to understand. In this study, we explore two specific types of feature representations: those obtained from the Scattering Transform, a designed representation, and those from MusicNN, a learned representation.

3.3.2 Designed vs. Learned Feature Representations

Designed feature representations, such as the Scattering Transform, are crafted based on theoretical principles and domain knowledge. They are engineered to capture specific characteristics of the data that are known to be informative for a given task. In contrast, learned feature representations, such as those obtained from MusicNN, are derived from data through a learning process, typically involving a neural network that learns to transform the raw data into a useful format as part of the training process.

3.3.3 Scattering Transform - We kinda talked already about this... Repetitive ?

The Scattering Transform is a designed feature representation that leverages wavelet transforms to produce translation-invariant representations of audio signals. It is constructed based on mathematical principles and aims to capture hierarchical structures in the data that are stable and informative.

3.3.4 MusicNN - Kinda short though? Perhaps longer more detail explanation?

MusicNN is a deep learning model that learns to extract meaningful features from raw audio data. As a learned feature representation, MusicNN is trained on a large dataset to understand and capture the relevant patterns in the data, which are then used as features for further analysis.

3.3.5 Summary

In summary, feature representations transform raw data into a format that machine learning algorithms can more effectively analyze. In this study, we compare two types of feature representations: designed, as exemplified by the Scattering Transform, and learned, as demonstrated by MusicNN.

3.4 Machine Learning Methods

3.4.1 Introduction

Machine learning methods are a cornerstone in the field of music information retrieval, enabling computers to learn from data and make predictions or decisions without being explicitly programmed to perform the task. This section provides an overview of three widely used machine learning methods: Support Vector Machines (SVM), k-Nearest Neighbors (KNN), and Random Forest. Each method is introduced with its theoretical background, practical applications, advantages, and disadvantages, aiming to equip the reader with a fundamental understanding of these techniques.

3.4.2 Support Vector Machines (SVM)

Support Vector Machines (SVM) are a set of supervised learning methods used for classification and regression tasks. They work by finding the hyperplane that best divides a dataset into classes [?].

Advantages

- Effective in high dimensional spaces.
- Maximizes margin, so it is robust to noise.

Disadvantages

- Can be sensitive to overfitting if the number of features is much greater than the number of samples.
- Computationally intensive, making it less suitable for large datasets.

Theory and Operation

Support Vector Machines (SVMs) are a class of supervised learning algorithms that are used for binary classification tasks. The fundamental idea behind SVMs is to construct a hyperplane in a high-dimensional feature space that distinctly separates the data points into two classes. Here is a more detailed explanation:

1. **Hyperplane:** In an n -dimensional space, a hyperplane is a flat affine subspace of dimension $n - 1$. In two dimensions, the hyperplane is a line, and in three dimensions, it is a plane. For SVMs, the hyperplane is defined as the set of points x such that $w \cdot x + b = 0$, where w is the weight vector, x is the input vector, and b is the bias term.
2. **Margin:** The margin is defined as the distance between the closest points (support vectors) of the two classes to the hyperplane. Mathematically, it is calculated as $\frac{2}{\|w\|}$, where $\|w\|$ is the Euclidean norm of the weight vector w . The objective of SVM is to maximize this margin.
3. **Support Vectors:** These are the data points that are closest to the hyperplane and are the most difficult to classify. They lie along the edges of the margin, and they essentially define the position of the hyperplane. In mathematical terms, support vectors are the points x for which $w \cdot x + b = \pm 1$.
4. **Kernel Trick:** For data that is not linearly separable in its original feature space, SVMs can employ a technique called the kernel trick. This involves mapping the data into a higher-dimensional space where it becomes linearly separable. Popular kernels include polynomial kernels, radial basis function (RBF) kernels, and sigmoid kernels.
5. **Optimization Problem:** The SVM algorithm involves solving a convex quadratic optimization problem. The objective is to find the weight vector w and bias term b that maximize the margin while correctly classifying all (or most, in the case of soft margin SVM) of the training

data. This is typically formulated as a constrained optimization problem and solved using techniques such as Sequential Minimal Optimization (SMO).

In summary, SVMs find the hyperplane that has the maximum margin between the support vectors of two classes. This hyperplane then acts as the decision boundary for future data points, classifying a new point based on which side of the hyperplane it falls on.

3.4.3 k-Nearest Neighbors (KNN)

The k-Nearest Neighbors (KNN) algorithm is a simple, instance-based learning method used for classification and regression [?].

Advantages

- Simple and easy to implement.
- Does not make any assumptions on the data distribution.

Disadvantages

- Computationally expensive, especially with large datasets.
- Sensitive to irrelevant features and the scale of the data.

Theory and Operation

The k-Nearest Neighbors (KNN) algorithm is a simple and intuitive supervised learning method used for classification and regression tasks. It classifies a data point based on how its neighbors are classified. Here is a more detailed explanation:

1. **Choosing k :** The first step in KNN is to choose the number of neighbors, k . This is a hyperparameter that determines how many neighbors will be considered when making a classification decision. A small k (e.g., 1) can be noisy and subject to outliers, while a large k smoothens the decision boundary but may blur the classes.

2. **Distance Metric:** KNN requires a distance metric to calculate the distance between data points. Common choices include Euclidean distance, Manhattan distance, and Minkowski distance. The choice of distance metric can significantly affect the performance of the algorithm.
3. **Finding Neighbors:** For a given data point that we want to classify, the algorithm calculates the distance between that point and every other point in the dataset. It then sorts these distances and selects the k smallest distances, identifying the k nearest neighbors.
4. **Majority Voting:** For classification tasks, KNN classifies the new data point based on a majority vote among its k nearest neighbors. That is, it assigns the class that is most common among the neighbors. For regression tasks, it can take the average of the k neighbors as the prediction.
5. **Handling Ties:** In some cases, there might be a tie in the majority voting. Various strategies can be used to break the tie, such as choosing the class of the closest neighbor among the ties, or randomly selecting a class among the tied classes.

In summary, KNN is a non-parametric method that works by comparing a given test sample with k training samples that are closest to it in the feature space. The output is a class membership, which is the most frequent class among the k nearest training samples.

3.4.4 Random Forest

Random Forest is an ensemble learning method that operates by constructing a multitude of decision trees during training time and outputting the class that is the mode of the classes of the individual trees [?].

Advantages

- Highly accurate and can handle large data sets with higher dimensionality.
- Can handle missing values and maintains accuracy for missing data.

Disadvantages

- Can be computationally intensive, thus requiring powerful machines.
- The model can be difficult to interpret compared to a decision tree.

Theory and Operation

Random Forest builds multiple decision trees and merges them together to get a more accurate and stable prediction.

3.4.5 Summary

This section introduced three widely used machine learning methods, each with its own strengths and weaknesses. SVMs are powerful for high-dimensional data but can be computationally intensive. KNN is simple and effective but may struggle with large datasets and irrelevant features. Random Forests are highly accurate and can handle large, complex datasets, but they may require significant computational resources and can be challenging to interpret. Each of these methods offers unique advantages depending on the specific task at hand.

3.5 Dimensionality Reduction Techniques

Dimensionality reduction techniques in machine learning deal with the problem of high-dimensional data. By transforming the original features into a new set of fewer and uncorrelated variables, these techniques aim to simplify the data's structure. This simplification can lead to more efficient storage and computation, easier visualization, and may help to improve the performance of machine learning models by reducing the risk of overfitting. Principal Component Analysis (PCA) is a classical method in this category, and in this section, we will delve into PCA and explore other common dimensionality reduction techniques.

3.5.1 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a statistical procedure that uses orthogonal transformations to convert correlated features (or variables) of a dataset into a set of linearly uncorrelated features called principal components. The first principal component explains the most variance in the data, and

each succeeding component (which is orthogonal to the preceding ones) explains a decreasing amount of variance. Mathematically, PCA seeks to find the eigenvalues and eigenvectors of the data's covariance matrix, and these eigenvectors define the new axes of the reduced feature space.

- **Advantages:** PCA is simple, non-parametric, and can be effective for linearly separable data. It is widely used for its ability to compress data efficiently and its effectiveness in removing noise.
- **Disadvantages:** One of the main drawbacks of PCA is that the principal components are often not interpretable. Additionally, it assumes that the principal components are a linear combination of the original features.

3.5.2 Other Dimensionality Reduction Techniques

t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a probabilistic technique that aims to preserve small pairwise distances or similarities between data points in a high-dimensional space in a low-dimensional embedding. It is particularly effective for visualizing high-dimensional data.

Linear Discriminant Analysis (LDA)

LDA is a supervised method that seeks to find the linear combinations of features that best separate two or more classes in a dataset. It is particularly useful when the classes are well-separated.

Autoencoders

Autoencoders are neural networks used for dimensionality reduction; they are trained to encode high-dimensional data into a lower-dimensional form and then decode it back. They are effective for complex and non-linear data.

Isomap

Isomap is a non-linear dimensionality reduction method that seeks to preserve geodesic (or 'curved') distances in the lower-dimensional embedding. It is particularly effective for data that lies on a manifold.

Chapter 4

Method or Methods

4.1 Model Training and Evaluation

4.2 Essid Dataset

The Essid dataset, utilized in the study, is a collection of single instrument recordings. This dataset was designed with the aim of capturing a maximum number of different recording conditions, performers, and types of music for each instrument. The extracts were obtained from digital recordings of classical music, jazz, or sound carriers used for teaching music, as well as studio recordings.

The Essid dataset was proposed by Slim Essid in his PhD thesis titled “Classification automatique des signaux audio-fréquences: reconnaissance des instruments de musique” in 2005. The thesis aimed to improve the automatic identification of instruments in realistic contexts.

The dataset used in the study includes seven instruments: guitar (Gt), piano (Pn), cello (Co), clarinet (Cl), oboe (Ob), trumpet (Tr), and violin (Vl). For each of these instruments, there are several 5-second snippets from recordings from five different recording conditions per class.

Here is a comprehensive overview of the Essid dataset:

Instrument	Code	Sources	Samples/source	Total samples
Guitar	Gt	5	74	370
Piano	Pn	5	77	385
Cello	Co	5	116	580
Clarinet	Cl	5	56	280
Oboe	Ob	5	90	450
Trumpet	Tr	5	49	245
Violin	VI	5	89	445
Total				2,755

Table 4.1: Overview of the Essid dataset

Instrument	Total Samples
Guitar	10
Piano	10
Cello	10
Clarinet	10
Oboe	10
Trumpet	10
Violin	10
Total	70

Table 4.2: Overview of the Youtube Excerpts Dataset

Chapter 5

What you did

5.1 Data Transformation Pipeline

5.1.1 Overview

In our experiments, we transform the raw audio samples from the ESSID dataset into feature representations that are conducive for various downstream tasks, such as classification or similarity analysis. We employ three different transformation techniques: the Scattering Transform, MusicNN with Million Song Dataset (MSD) weights, and MusicNN with MagnaTagATune (MTT) weights.

5.1.2 Scattering Transform

Computation of Scattering Coefficients

For each audio sample in the ESSID dataset, we compute the scattering coefficients. These coefficients are high-dimensional tensors that encapsulate the hierarchical structure of the audio signal, capturing intricate details at various scales and orders.

Logarithmic Scaling

To stabilize the variance and make the coefficients more amenable to statistical analysis, we apply logarithmic scaling. Specifically, we add a small epsilon value (e.g., $1e - 6$) to the absolute value of the coefficients and then take the natural logarithm:

$$Sx_{\text{scaled}} = \log(|Sx| + \epsilon)$$

Temporal Averaging

We take the mean over the time dimension of the scaled coefficients:

$$Sx_{\text{mean}} = \text{mean}(Sx_{\text{scaled}}, \text{axis} = -1)$$

Why Temporal Averaging?

Temporal averaging is used to create a summary statistic that captures the average behavior of each scattering coefficient over time for each audio sample.

Resulting Dimensionality

After these preprocessing steps, for each audio sample in the ESSID dataset, we obtain a feature vector of dimension 373.

5.1.3 MusicNN Transforms

For the MusicNN MSD and MTT models, we transform the raw audio samples from the ESSID dataset into feature representations by extracting the activations from the penultimate layer of the pretrained convolutional neural networks.

5.2 Experiments

Chapter 6

Results and Analysis

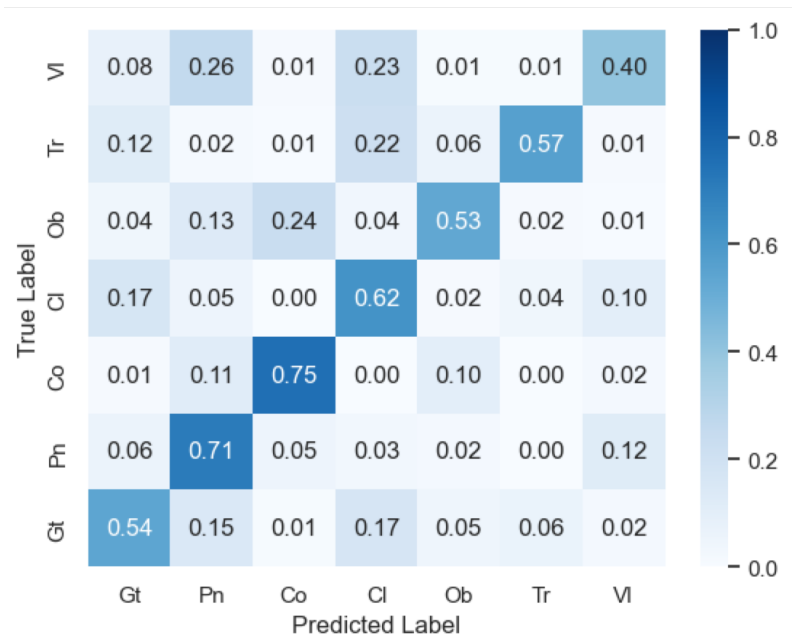


Figure 6.1: KNN Performance on PF_MSD_ESSID Dataset

We will modify this text as well as add better captions. Perhaps add F1, Recall, Precision and accuracy as well? Perhaps in a table or will that get messy? Based on these confusion matrices, we clearly see that a concatenation of Sx and MTT grants a better result. However, we can't say for sure why (yet). This is due to the fact that the dimensions of the different spaces hasn't been taken into account. As mentioned before, Sx, MTT and their combined representation is 200, 373 and 573 features per sample, hence, it could be

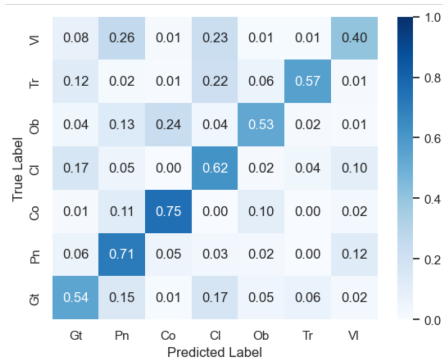


Figure 6.2: KNN Performance on MSD_ESSID Dataset



Figure 6.3: SVM Performance on MSD_ESSID Dataset

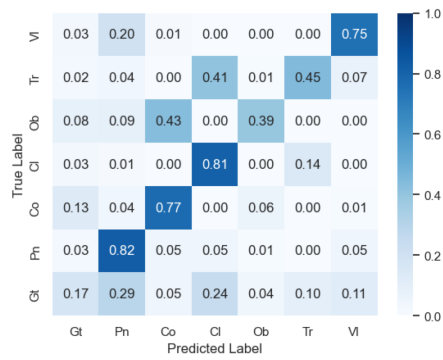


Figure 6.4: KNN Performance on sx mean features



Figure 6.5: SVM Performance on sx mean features

that the higher dimensionality is the reason for the increase in performance. Results below will help us though...

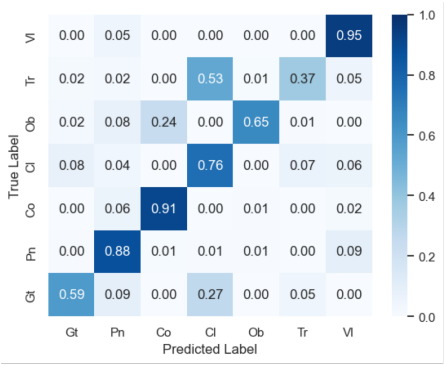


Figure 6.6: KNN Performance on combined features, sxmean and mtt



Figure 6.7: SVM Performance on combined features, sxmean and mtt



Figure 6.8: SVM on msd (youtube), for 200 dim space



Figure 6.9: SVM on msd (youtube), for 200 dim space, but projected onto the pca of the msd space

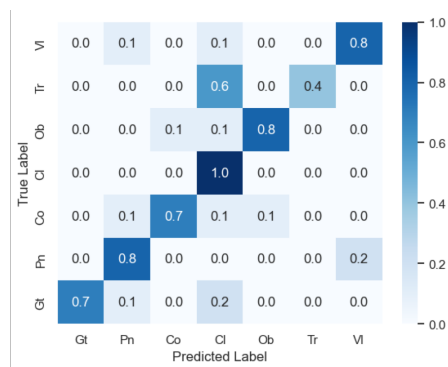


Figure 6.10: SVM on combined (youtube dataset), for 573 dim space

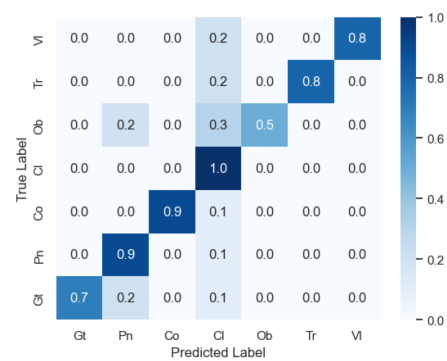


Figure 6.11: SVM on combined (youtube dataset), for 200 dim space. So this is projected onto the 200 PCs of the svm essid space

6.1 Dimensionality Reduction Using Principal Component Analysis

To compare the performance of different feature extraction methods (e.g., Scattering Transform and MusicNN) in a fair manner, we employ Principal Component Analysis (PCA) to reduce the dimensionality of the feature spaces to a common lower-dimensional subspace. This section outlines the steps involved in this process.

1. **Standardization of Features:** We start by standardizing the feature vectors. This involves scaling the features so that they have a mean of zero and a standard deviation of one.

$$X_{\text{standardized}} = \frac{X - \mu}{\sigma} \quad (6.1)$$

where X is the feature matrix, μ is the mean of the features, and σ is the standard deviation of the features.

2. **Computing the Principal Components:** We fit the PCA model to the standardized features of the training data from the ESSID dataset. This involves computing the eigenvectors and eigenvalues of the covariance matrix of the standardized features. The eigenvectors define a new set of orthogonal axes (the principal components), and the eigenvalues indicate the explained variance along each axis.
3. **Selecting the Number of Components:** For a fair comparison between different feature extraction methods, we reduce the dimensionality of the Scattering Transform features from 373 to 200, matching the dimensionality of the MusicNN features.
4. **Projection onto Lower-Dimensional Subspace:** We transform the standardized features by projecting them onto the selected principal components. This step effectively reduces the dimensionality of the data while retaining most of the variance.

$$X_{\text{reduced}} = X_{\text{standardized}} \cdot W \quad (6.2)$$

where X_{reduced} is the lower-dimensional feature matrix, and W is the matrix whose columns are the selected eigenvectors.

5. **Applying the Same Transformation to New Data:** For our YouTube excerpts dataset, we apply the same PCA transformation that was fit on the ESSID training data. This ensures that the new data is projected onto the same lower-dimensional subspace created by the PCA model.

$$X_{\text{new, reduced}} = (X_{\text{new}} - \mu) \cdot W \quad (6.3)$$

where X_{new} is the feature matrix of the new dataset.

This process ensures that the comparisons between different feature extraction methods are conducted in a common lower-dimensional space, thereby enabling a fair evaluation of their performance.

Chapter 7

Discussion

Chapter 8

Conclusions and Future work

Chapter 9

Something Extra

svensk: Extra Material som Bilaga

For DIVA

```
{
  "Author1": {
    "Last name": "Löfvenmark",
    "First name": "Niclas",
    "Local User Id": "u100001",
    "E-mail": "niclaslo@kth.se",
    "ORCID": "0000-0002-00001-1234",
    "organisation": {"L1": "School of Electrical Engineering and Computer Science ",
                    }
  },
  "Degree": {"Educational program": "Degree Programme in Engineering Physics"},
  "Title": {
    "Main title": "Comparative Analysis of Learned and Designed Features: ",
    "Subtitle": "A Deep Dive into MusicNN and Scattering Transform for Music Instrument Recognition",
    "Language": "eng" },
  "Alternative title": {
    "Main title": "",
    "Language": "swe"
  },
  "Supervisor1": {
    "Last name": "Sturm",
    "First name": "Bob",
    "Local User Id": "u100003",
    "E-mail": "sa@kth.se",
    "organisation": {"L1": "School of Electrical Engineering and Computer Science ",
                    "L2": "Computer Science" }
  },
  "Examiner1": {
    "Last name": "Holzapfel",
    "First name": "André",
    "Local User Id": "u100004",
    "E-mail": "holzap@kth.se",
    "organisation": {"L1": "School of Electrical Engineering and Computer Science ",
                    "L2": "Computer Science" }
  },
  "Other information": {
    "Year": "2026", "Number of pages": "xv,34"
  }
}
```